# Task 1

## Why We Knowingly Deviated from the Literal Requirements:

The assignment describes SPP as **two separate classifiers** bolted onto one IP: a uniform-stride detector (constant delta) and a non-uniform-stride detector (a repeating signature => next-stride table), with explicit priority/fallback logic between them.

Our implementation instead models **both patterns as a single generalized structure**: a per-IP graph of strides, where each observed stride is a node, and the edges between nodes (`next_strides`) record which stride tends to follow which. Concretely:

- A **uniform stride** (A, A+2, A+4, A+6…) shows up as a stride node with a self-loop: stride `+2` is always followed by stride `+2`. The DFS lookahead just keeps re-visiting the same node.
- A **non-uniform repeating pattern** (+1, +2, +2, +2, +3, repeat) shows up as a cycle through multiple stride nodes: `+1 -> +2 -> +2 -> +2 -> +3 -> +1 -> …`. The DFS lookahead walks this cycle exactly as the per-IP access stream does.

So instead of writing two separate detection paths with an explicit priority/fallback switch (uniform first, else non-uniform), we unified them into one data structure where uniform strides are simply the degenerate (single-node, self-looping) case of the general stride-transition graph. The LRU-sorted edge traversal prioritizes the most recently reinforced transitions, whether they are self-loops or cross-edges.

## Design Logic

**Tracker table (per-IP state).** Like IP-stride, up to `IP_TRACKER_COUNT = 24` IPs are tracked, masked to 8 bits (`MASK_IP`) and LRU-replaced. Each `Tracker_Element` keeps `last_cl_addr` (for computing the next stride), `last_stride` (a pointer to the stride node just seen), and a two-field prediction record (`predicted_stride`, `has_prediction`) used only to grade the *previous* prediction once the real next address arrives.

**Stride nodes instead of one scalar stride.** IP-stride keeps a single `last_stride` and only prefetches when the same stride repeats back-to-back  one scalar, one pattern. We replace that scalar with a small LRU table of up to `PAST_STRIDES = 7` distinct stride values per IP (`Tracker_Element::strides`). An IP that interleaves a few different strides (e.g. two co-resident array walks sharing one loop IP) gets a node per stride instead of having each new stride evict the only slot.

**Edges carry the pattern.** Every stride node owns its own small table (`next_strides`, 3 slots): which stride value followed it, and how often that specific transition has been confirmed, in a

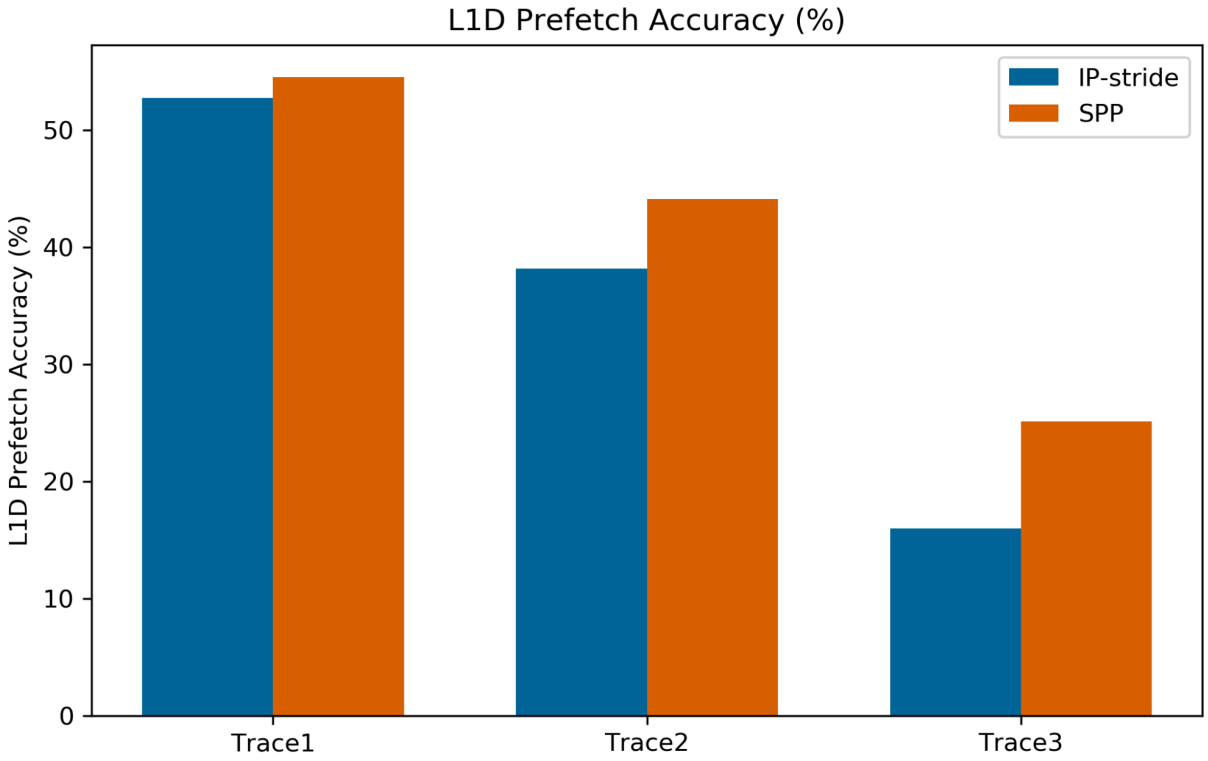
2-bit saturating counter (`confidence`, 0-3). This edge *is* the signature the spec asks for instead of hashing the last few strides into one key that maps to a predicted next stride, the previous stride value itself is the key, and it indexes directly into its own 3-entry forward table. A uniform stride shows up as a self-loop (stride *S* -> *S*) with confidence that quickly saturates; a repeating non-uniform sequence shows up as a chain of distinct nodes, each with a high-confidence edge to the next stride in the cycle. Nothing in the code has to decide which case it is looking at  the graph shape is the classification.

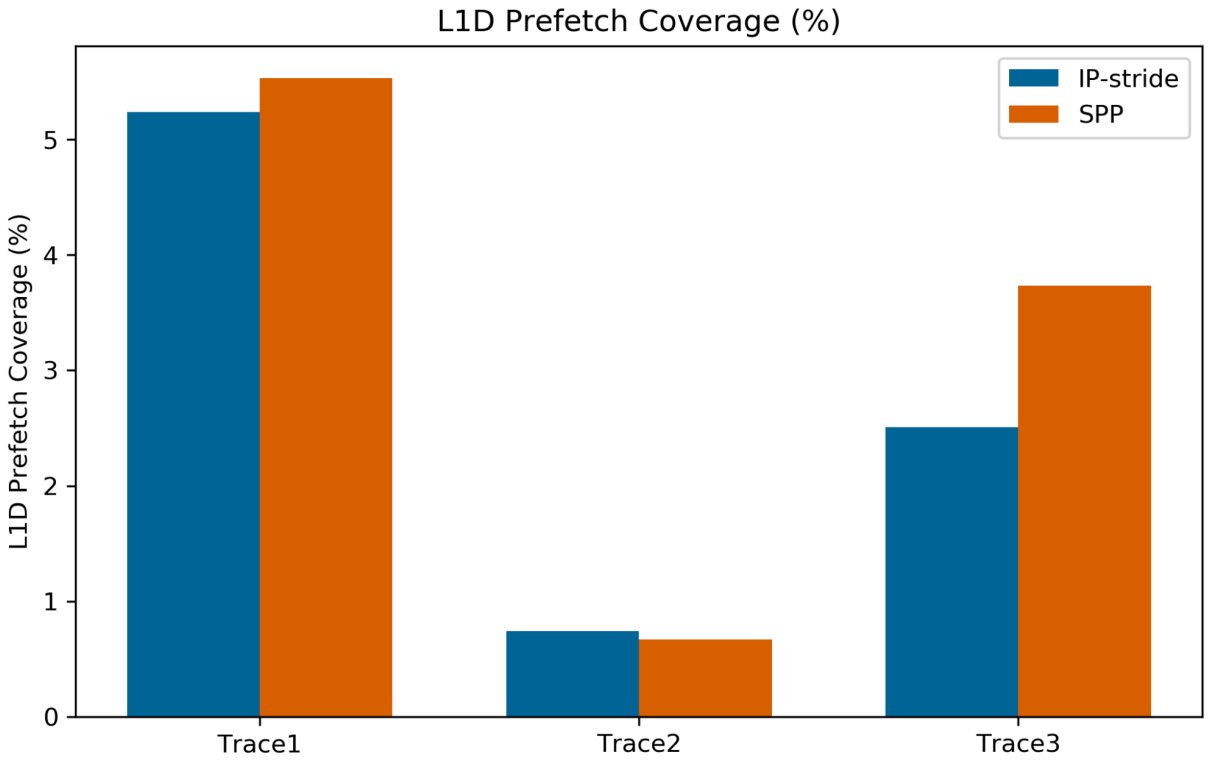
**Grading a prediction before making the next one.** On every access, before touching the current stride, the tracker checks whether the stride it predicted last time (`predicted_stride`) actually happened. If so, the specific edge responsible gets `confidence++`; if not, that edge alone gets `confidence--`. Only the edge that was wrong is punished; an IP with one unreliable transition among several reliable ones does not have its good predictions throttled by the bad one, which is the main advantage of per-edge over a single per-IP accuracy counter.

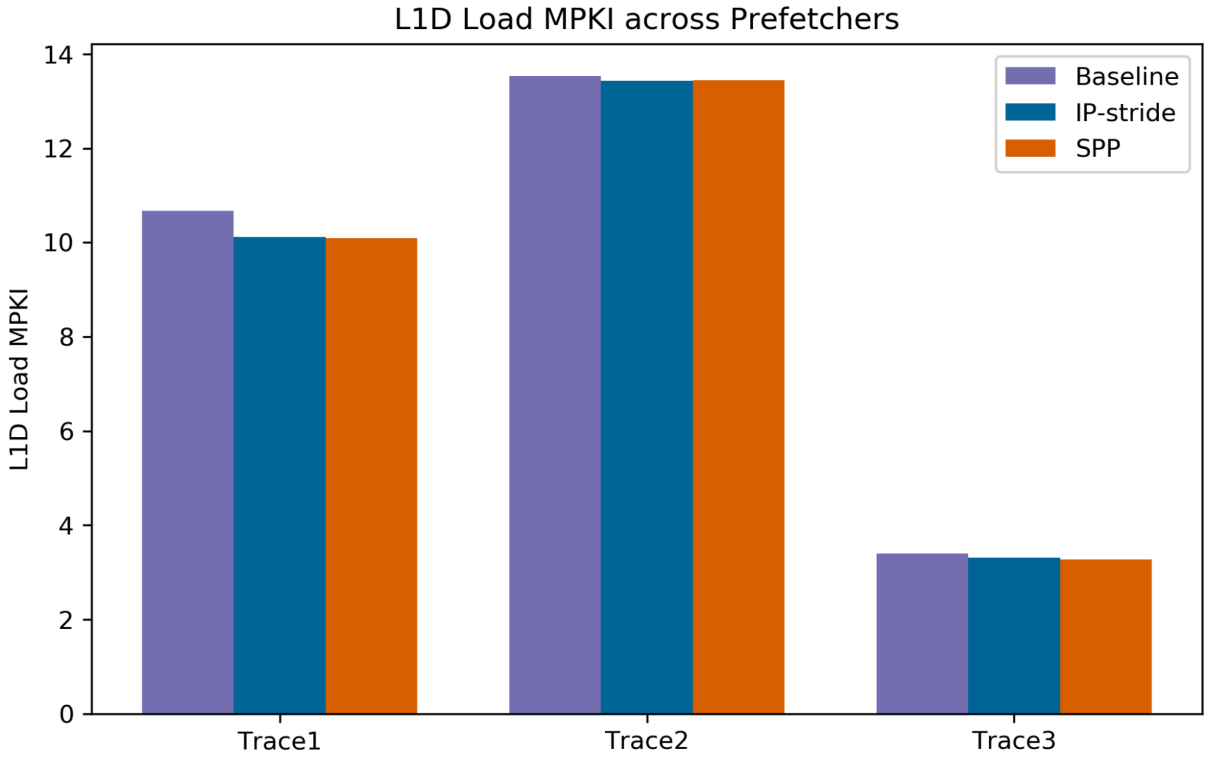
**DFS lookahead generates the prefetches.** Starting from the stride node just observed, `dfs_lookahead` sorts its outgoing edges by LRU (most recently confirmed first), converts each edge's confidence into a degree (`DEGREE_HIGH/MED/LOW`, section 6), and recurses into whichever successor node exists in the IP's stride table  this is what lets the prefetcher walk several hops ahead through a cycle in one shot rather than confirming one stride at a time, which is how a 5-element repeating pattern still gets predicted multiple steps ahead despite only ever storing one-hop edges. Recursion stops at a page boundary, when the degree budget is spent, or when no matching successor node exists (natural backtracking, since the loop simply continues to the next candidate edge).

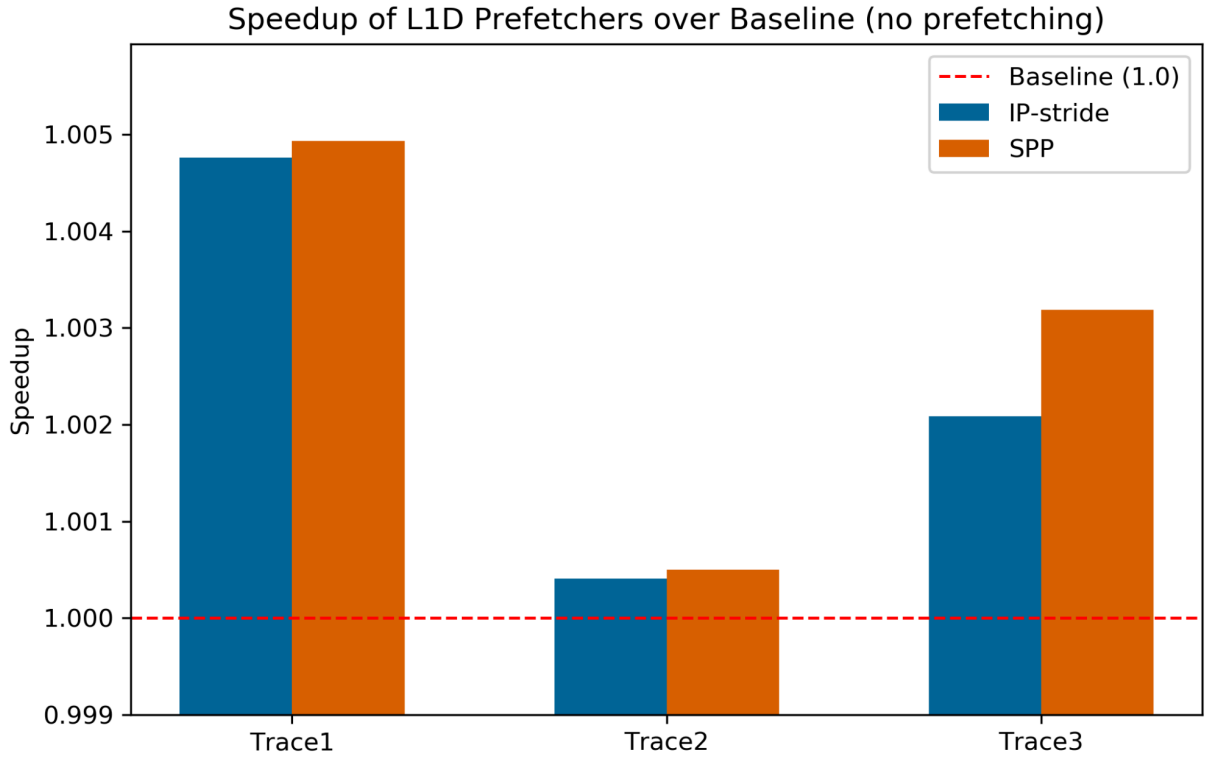
**Everything is LRU-bounded.** The IP table, each IP's stride table, and each stride's edge table are all fixed-size LRU structures, so total storage is bounded regardless of how irregular a workload's address stream is

## Results


L1D Prefetch Accuracy (%)
IP-stride
SPP
L1D Prefetch Accuracy (%)
0
10
20
30
40
50
Trace1
Trace2
Trace3


L1D Prefetch Coverage (%)
IP-stride
SPP
L1D Prefetch Coverage (%)
0
1
2
3
4
5
Trace1
Trace2
Trace3


L1D Load MPKI across Prefetchers
L1D Load MPKI
0
2
4
6
8
10
12
14
Trace1
Trace2
Trace3
Baseline
IP-stride
SPP


Speedup of L1D Prefetchers over Baseline (no prefetching)
Speedup
0.999
1.000
1.001
1.002
1.003
1.004
1.005
Trace1
Trace2
Trace3
Baseline (1.0)
IP-stride
SPP

### Storage Overhead

| Field | Width | Why |
|---|---|---|
| IP tag (`ip & MASK_IP`) | 8 bits | `MASK_IP = 0xFF` |
| Tracker LRU | 5 bits | $log_2 24$, 24 trackers |
| `last_cl_addr` | 58 bits | 64-bit address − 6-bit block offset |
| `last_stride` pointer -> index | 4 bits | $log_2 7$ index into the 7-slot stride table + 1 valid bit |
| `predicted_stride` | 8 bits | same signed-stride width |
| `has_prediction` | 1 bit | flag |
| **Tracker_Element fixed total** | **84 bits** | sum of the above |

| Field | Width | Why |
|---|---|---|
| Stride value | 8 bits | signed stride, page-bounded |
| Node LRU (of 7 slots) | 3 bits | $log_2 7$, `PAST_STRIDES` |
| Node valid bit | 1 bit | `LRU_Block` allows empty slots |
| **Stride node total (×7 per tracker)** | **12 bits × 7 = 84 bits** | |

| Field | Width | Why |
|---|---|---|
| Next-stride value (edge target) | 8 bits | signed stride |
| Edge LRU (of 3 slots) | 2 bits | $log_2 4$, `MAX_PREFETCH DEGREE` |
| Edge confidence | 2 bits | `CONF_MIN`=0…`CONF_MAX`=3 |
| Edge valid bit | 1 bit | `LRU_Block` allows empty slots |
| **Edge total (×3 per node, ×7 nodes per tracker)** | **13 bits × 21 = 273 bits** | |

Per tracker: 84 + 84 + 273 = 441 bits.
Across 24 trackers: **441 × 24 = 10,584 bits ≈ 1.29 KB**.

## Accuracy-Based Throttling: Threshold Choice

We throttle **per edge**, not per IP: every stride-to-stride transition (confidence) carries its own 2-bit saturating counter, 0-3, incremented when the transition it predicted actually occurred and decremented otherwise. A global, per-IP accuracy counter was the more literal reading of the assignment, but it punishes every prediction an IP makes whenever any one of its transitions is wrong, even if the other transitions for that same IP are reliable; per-edge throttling isolates the cost of being wrong to the one transition that was actually wrong, and it reuses a field the design already needed for the Markov edge itself, so it adds no storage beyond the 2 bits counted.

The threshold split is

- `confidence >= CONF_HIGH_THRESH (2)` -> aggressive,
- `confidence <= CONF_LOW_THRESH (1)` -> minimal.

With a 2-bit counter those two conditions already cover every possible value (0,1 -> low; 2,3 -> high), so `DEGREE_MED` is currently unreachable  it is kept in the code as a clean middle tier for if the counter is ever widened past 2 bits, rather than hard-coding a 2-way split into the comparison logic.

We picked the midpoint of the 2-bit range as the cut rather than, say, requiring `confidence == CONF_MAX` for aggressive treatment, for two reasons:

- **A 2-bit saturating counter needs at least two wrong guesses in a row to fall out of the trusted band**, not one. An edge sitting at `confidence = 3` after one misprediction drops to 2, which is still "aggressive"  a single coincidental miss does not immediately kill an otherwise-good transition, avoiding thrash between degrees on noisy but mostly-correct patterns.
- **An edge still has to earn two net-correct predictions from a cold start `(confidence = 0`) before it is trusted aggressively.** That is enough evidence to rule out a one-off coincidental match (which IP-Stride's "repeat the same stride twice" rule is also implicitly relying on) while being cheap to reach  a well-behaved uniform or repeating pattern clears it within one or two repetitions of its cycle.

## Choice of Prefetch Degrees

Three degree tiers map from an edge's confidence: `DEGREE_HIGH = 4, DEGREE_MED = 3, DEGREE_LOW = 1`

- `DEGREE_LOW = 1` matches an untrusted or cold edge to essentially IP-Stride's own behavior before it has confirmed a repeat (IP-Stride issues nothing until it has seen the same stride twice; we allow exactly one prefetch on an unconfirmed edge, which is as conservative as IP-Stride gets while still making forward progress). This bounds the cache and bandwidth pollution a wrong guess can cause to a single line.

- `DEGREE_MED = 3` mirrors IP-Stride's own fixed degree (`PREFETCH_DEGREE = 3` in the reference code), so a moderately-confident transition gets exactly the aggressiveness the baseline always uses unconditionally  our design only grants that level once an edge has cleared the low-confidence band, rather than giving it away for free on every repeat like IP-Stride does.
- `DEGREE_HIGH = 4` goes one step past IP-Stride's baseline because a high-confidence edge has already demonstrated repeatability beyond what IP-Stride requires to prefetch at all; we judged that one extra line of lookahead is worth the small added MSHR/bandwidth pressure once an edge has earned that much trust, without pushing degree so high that a single misprediction wastes many lines at once.

The budget also shrinks with recursion depth: `dfs_lookahead` passes `effective_budget = min(degree_budget, edge_degree)` into each recursive call, so the degree granted at hop *N+1* can never exceed what hop *N* allowed, and a confident first hop followed by a shaky second hop is capped by the weaker of the two rather than by whichever edge happens to be checked first. This keeps the DFS from using one strong edge near the root of the walk to justify unlimited aggressiveness several hops of uncertainty later  the lookahead window naturally contracts as confidence falls off down the chain, which is what lets `DEGREE_HIGH = 4` stay a safe ceiling: it only pays out in full along a chain of transitions that have each individually earned it.

# Task 2

## Task 2.a

### Introduction

Conventional replacement policies such as Re-Reference Interval Prediction (RRIP) primarily estimate the likelihood that a cache line will be reused. However, a conventional policy does not necessarily distinguish between instruction lines and data lines. Consequently, instruction lines that are important for frontend progress may be evicted even when data lines could be selected instead.

To address this limitation, this assignment implements Instruction RRIP (I-RRIP) at the L2 cache. The policy retains the RRIP replacement framework while incorporating instruction awareness into cache-line insertion, promotion, and victim selection. The primary objective is to reduce L2 instruction-load misses per thousand instructions (instruction MPKI), while also evaluating the effects on overall IPC, speedup, total L2 load MPKI, and data-load MPKI.

### Design Overview

The implementation uses a two-dimensional RRPV array and an additional array to track the type of each resident cache line:

> `L2_rrpv[set][way]`, stores the Re-Reference Prediction Value (RRPV) of each cache line. (Already part of old RRIP model)
>
> `L2_is_instruction[set][way]`, records whether the resident cache line contains instructions. (Newly added as part of the assignment)

| Event | Instruction line | Data line |
|---|---|---|
| Insertion after a miss | RRPV = 1 | RRPV = 2 |
| Promotion after a hit | RRPV = 0 | RRPV = 1 |
| Victim preference when RRPV = 3 | Prefer retaining it | Prefer evicting it |

The last row describes the selection preference, not an unconditional rule: a data line is selected first among eligible lines, while an instruction line can still be evicted when no eligible data line is available.

This design gives instruction lines a more favorable replacement state without making them permanently resident or completely immune to eviction.

# Intuition

## Insertion:

Insertion determines the initial RRPV assigned to a cache line when a miss causes the line to enter the cache. This decision matters because a newly inserted line may compete with existing lines for limited cache capacity. In the implementation, instruction lines are inserted with RRPV 1, whereas data lines are inserted with RRPV 2. Since the maximum RRPV is 3, instruction lines begin farther from the replacement threshold than data lines.

Instruction lines are assigned a lower initial RRPV because preserving them in L2 can help reduce the latency associated with subsequent L1 instruction misses. When an instruction line is evicted prematurely, a later instruction access may need to obtain the line from a lower level of the memory hierarchy. This can increase instruction-fetch latency and negatively affect frontend progress.

By inserting instruction lines at RRPV 1 rather than RRPV 2, I-RRIP makes instruction lines less immediately vulnerable to replacement. A data line inserted at RRPV 2 is closer to the maximum RRPV and therefore closer to becoming eligible for eviction.

This distinction is particularly useful when the instruction working set exceeds L1I capacity and frequently accessed instructions must be retained in L2.

An alternative insertion policy would insert instruction lines at RRPV 2 and data lines at RRPV 3. We rejected it because data lines inserted at the maximum RRPV become eviction candidates immediately, so they are evicted before they get a chance to be reused, potentially causing excessive data-line eviction and unnecessary refetching.

## Promotion:

Promotion takes place whenever a cache-line access hits within the L2 cache. This hit provides concrete evidence of line reuse, prompting the replacement scheme to refresh the line's corresponding RRPV.

Under our implemented policy, hitting on an instruction line updates its RRPV to 0, whereas hitting on a data line updates its RRPV to 1:

- Instruction hit, RRPV = 0

- Data hit, RRPV = 1

An instruction hit indicates that the corresponding instruction line remains useful to the processor. Setting its RRPV to 0 gives it the strongest protection in this implementation against immediate replacement.

This choice complements the insertion policy. Instruction lines already enter the cache with a lower RRPV, a subsequent hit reinforces their retention by resetting the value to 0.

The objective is to preserve instruction lines that are actively being reused, thereby increasing the probability that later instruction accesses can be served by L2 rather than by a lower cache level or memory.

A data-line hit reduces its RRPV to 1, indicating that the line has demonstrated reuse and should not be treated as immediately disposable.

This is important because data accesses also contribute to application performance. A policy that never promoted data lines could evict frequently reused data and increase data MPKI, potentially offsetting the benefit of reducing instruction misses.

The difference between RRPV 0 for instructions and RRPV 1 for data expresses the policy's priority, both line types benefit from a hit, but instruction lines receive stronger protection.

The promotion policy makes instruction awareness responsive to actual cache accesses. It does not rely only on a line's initial insertion state, it also uses subsequent reuse to maintain instruction line priority.

## Eviction:

The eviction policy determines which cache line is removed when a new line must be allocated and the target set has no invalid way available.

In standard RRIP, the policy searches for a line whose RRPV equals the maximum value, 3. If no such line exists, the RRPVs are incremented until a replacement candidate becomes available. I-RRIP retains this basic RRIP victim-search and aging framework. However, it introduces instruction awareness when multiple lines are eligible for replacement.

When the victim-selection procedure encounters a line with RRPV 3, it checks whether that line is an instruction line or a data line. If the eligible line is a data line, it is returned as the victim immediately. If the eligible line is an instruction line, the implementation records it as a fallback candidate and continues searching for an eligible data line.

Consequently, when both instruction and data lines have RRPV 3, the policy prefers to evict a data line. This gives instruction lines additional protection at the final replacement decision, even when their RRPVs indicate that they are eligible for eviction.

This behavior is important because insertion and promotion biases alone do not guarantee that data lines will be selected first when several lines reach the maximum RRPV. Explicitly checking the line type at victim selection makes the intended instruction preference apply at the actual point of eviction.

## Why this eviction strategy is appropriate

The eviction rule complements insertion and promotion. Insertion gives instruction lines a better starting RRPV, promotion strengthens their protection after a hit, and victim selection favors data lines when both types are equally eligible for replacement.

This combination avoids relying on a single mechanism to achieve instruction prioritization. It also avoids making instruction lines completely immune to replacement.

Preferentially retaining instructions can reduce instruction MPKI, but it may also increase data MPKI if useful data lines are evicted more frequently. But according to the project the main metric that we were targeting was Instruction LOAD MPKI, and the assignment has shown significant results for Instruction LOAD MPKI using this policy.

# Experimental Results and Analysis

We evaluate the proposed I-RRIP policy against the baseline replacement policy using IPC, speedup, and L2 cache load MPKI. The evaluation considers total L2 load MPKI, instruction load MPKI, and data load MPKI to determine whether prioritizing instruction lines improves frontend performance and what effect it has on data accesses.

| config | trace | IPC | Speedup over LRU | Speedup over RRIP | L2C Load MPKI | L2 Instruction Load MPKI | L2 Data Load MPKI |
|---|---|---|---|---|---|---|---|
| LRU | trace1 | 0.23234 | 1 | 1.00049 | 17.6956 | 10.3377 | 7.35796 |
| LRU | trace2 | 0.25116 | 1 | 0.99743 | 27.3568 | 18.8533 | 8.50352 |
| LRU | trace3 | 0.38206 | 1 | 1.00085 | 3.95424 | 1.12408 | 2.83016 |
| LRU | AVG | 0.28852 | 1 | 0.99959 | 16.33555 | 10.10503 | 6.23055 |
| RRIP | trace1 | 0.23223 | 0.99951 | 1 | 17.6387 | 10.5618 | 7.07688 |
| RRIP | trace2 | 0.25181 | 1.00258 | 1 | 25.7458 | 17.7813 | 7.96452 |
| RRIP | trace3 | 0.38174 | 0.99915 | 1 | 4.03948 | 1.19704 | 2.84244 |
| RRIP | AVG | 0.28859 | 1.00041 | 1 | 15.80799 | 9.84671 | 5.96128 |
| I-RRIP | trace1 | 0.23224 | 0.99954 | 1.00003 | 17.615 | 9.90608 | 7.70892 |
| I-RRIP | trace2 | 0.25637 | 1.02075 | 1.01813 | 22.353 | 13.2912 | 9.06188 |
| I-RRIP | trace3 | 0.38198 | 0.99976 | 1.00062 | 3.8994 | 1.02368 | 2.87572 |
| I-RRIP | AVG | 0.29019 | 1.00669 | 1.00626 | 14.62247 | 8.07365 | 6.54884 |

## IPC Comparison:

This plot compares overall processor throughput under the two replacement policies. An IPC improvement would indicate that the benefits of instruction-line retention translate into better execution performance. A reduction in instruction MPKI alone does not guarantee higher IPC, since other cache misses and processor bottlenecks may also affect execution. It is visible from the plot that the new I-RRIP policy is providing either comparable or better IPC compared to the other two policies.

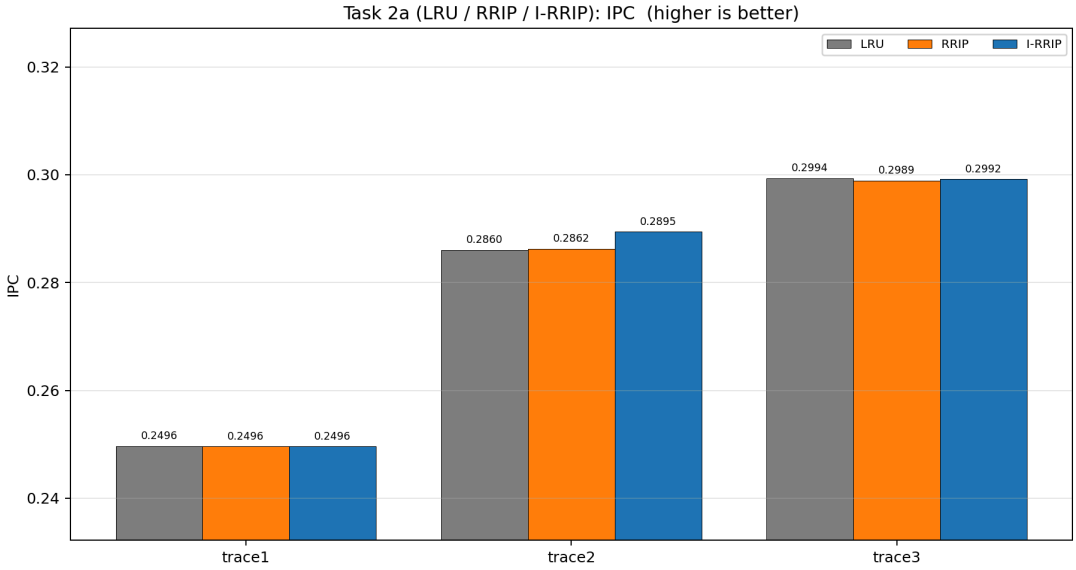



**Figure 1. IPC comparison between LRU, RRIP and I-RRIP across traces.**

## Speedup relative to LRU:

Trace 2 shows a clear speedup, which suggests it is an instruction-heavy workload that benefits from I-RRIP's prioritization. Trace 3 performs slightly worse than LRU, which suggests a more data-heavy workload where I-RRIP is not the best choice. Trace 1 is roughly on par with LRU.

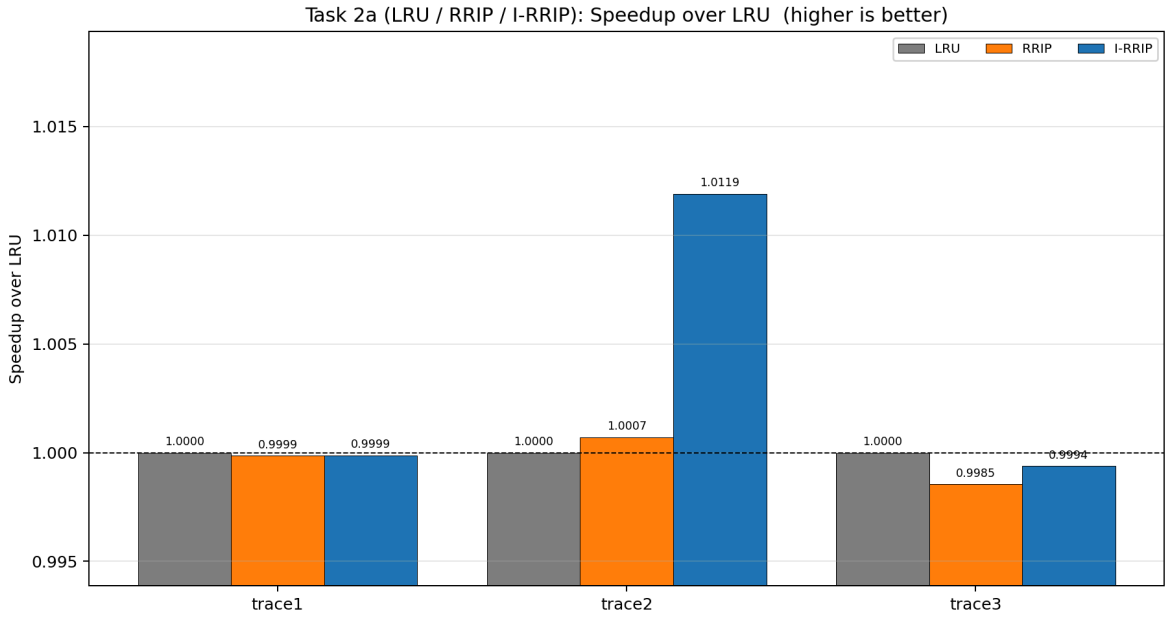



**Figure 2. Speedup of I-RRIP relative to LRU.**

## Speedup relative to RRIP:

The results show that the effectiveness of I-RRIP varies across workloads. For trace1, I-RRIP achieves approximately the same performance as RRIP, with both policies reporting a normalized speedup of 1.0000. For trace2, I-RRIP achieves a speedup of 1.0181, corresponding to an approximately 1.12% IPC improvement over RRIP. This is the largest observed gain and suggests that instruction aware cache replacement may benefit workloads whose performance is sensitive to instruction-line retention. For trace3, I-RRIP provides only a marginal improvement of approximately 0.08%.

The results indicate that prioritizing instruction lines does not consistently improve performance across all workloads.

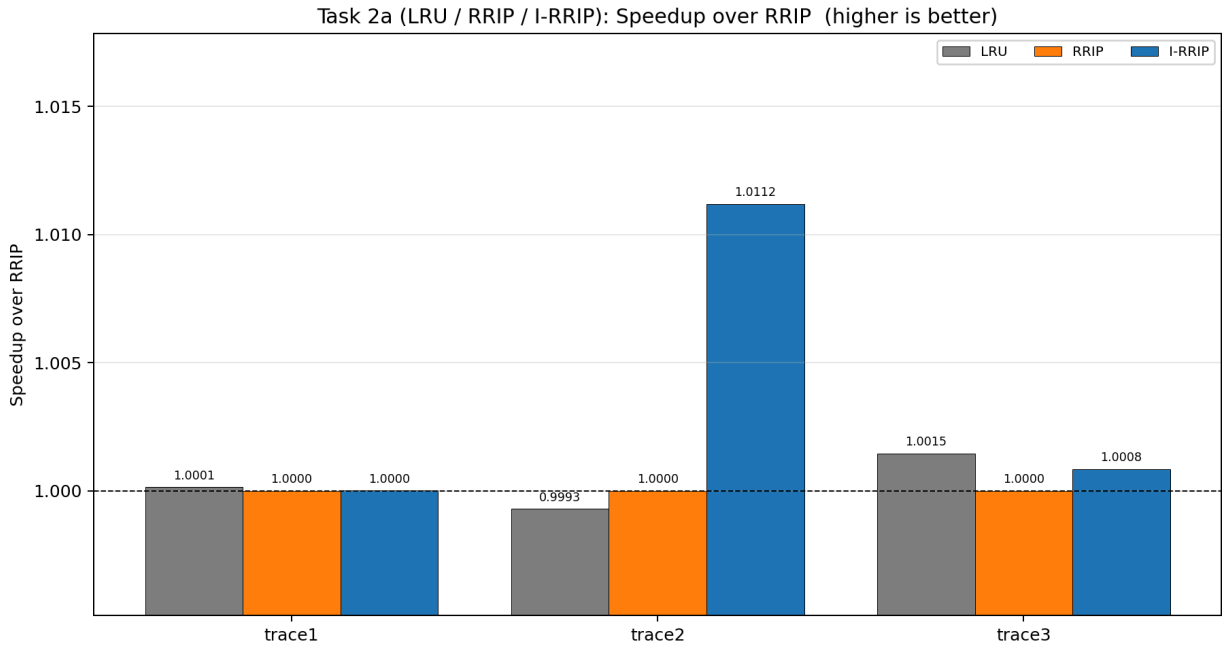



**Figure 3. Speedup of I-RRIP relative to RRIP.**

## Total L2 Load MPKI:

From the plot, it can be seen that across all the traces, prioritizing Instruction Lines have reduced Total Load MPKI. This observation is consistent with the policy's design, which gives instruction lines greater retention priority through more favorable insertion and promotion RRPV values and prefers data lines as eviction candidates. However, the impact on execution performance varies across workloads. In particular, trace2 shows the largest IPC improvement over RRIP, whereas trace1 and trace3 exhibit negligible gains. This indicates that reducing cache misses alone does not guarantee a proportional increase in IPC.

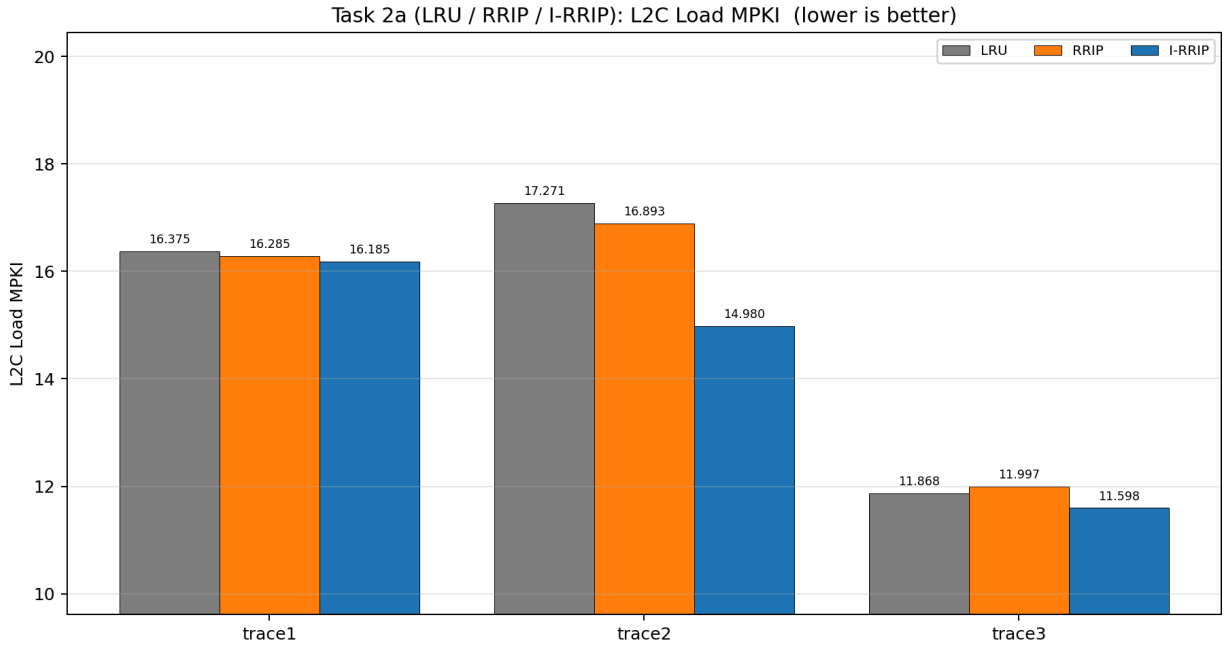



**Figure 4. Total L2 Load MPKI comparison**

## L2 Instruction MPKI:

Since the proposed policy explicitly prioritizes instruction lines during cache replacement, we expected it to reduce Instruction MPKI compared to conventional LRU and RRIP policies. The results presented in Figure 5 strongly support this hypothesis, as I-RRIP achieves significantly lower Instruction MPKI across all three traces than both baseline policies. By retaining instruction lines for longer, I-RRIP can potentially reduce instruction-fetch misses and the associated delays in supplying instructions to the processor. These results demonstrate that incorporating instruction aware replacement decisions can improve instruction cache behavior at the L2 level. However, the extent to which this reduction translates into overall performance improvement depends on the characteristics of each workload and the impact of the remaining cache misses on execution.

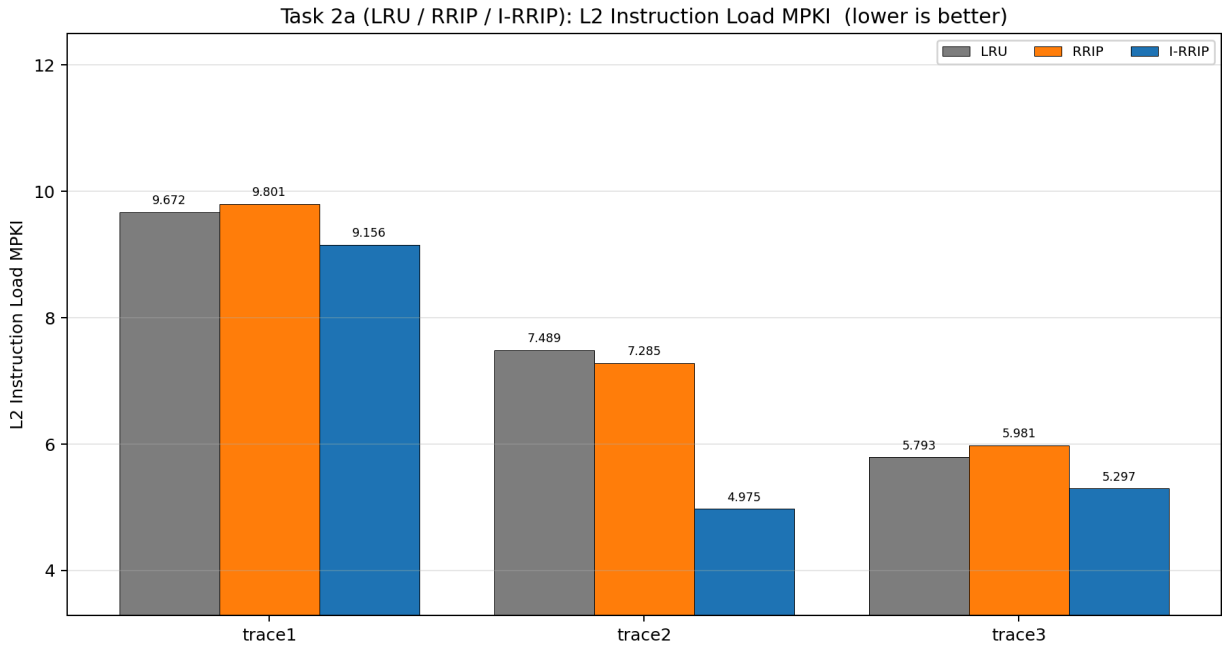



**Figure 5. L2 Instruction MPKI comparison**

## L2 Data MPKI:

As expected, prioritizing instruction lines leads to an increase in Data MPKI, and this trend is consistent across all three traces. This suggests that retaining instruction lines for longer comes at the cost of increased data cache misses, as data lines are more likely to be selected for eviction. Consequently, the improvement in Instruction MPKI is partially offset by the increase in Data MPKI, limiting the overall performance gains. This trade-off helps explain why I-RRIP achieves only marginal IPC improvements in most traces, despite the significant reduction in Instruction MPKI. The results highlight the importance of balancing instruction and data retention when designing an effective L2 cache replacement policy.

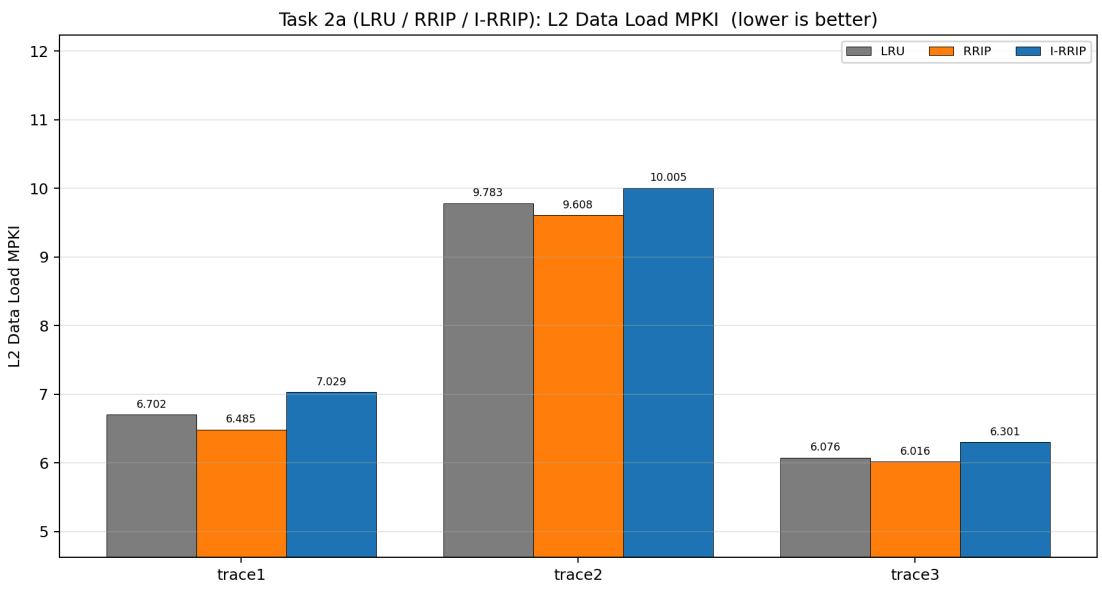



**Figure 6. L2 Data MPKI comparison**

# Conclusion

In this task, we implemented I-RRIP, an instruction-aware variant of RRIP that prioritizes instruction lines during L2 cache replacement. The policy assigns more favourable RRPV values to instruction lines during insertion and promotion, while preferring data lines as eviction candidates. The experimental results demonstrate that this approach successfully reduces L2 Instruction MPKI across all three traces compared to both LRU and conventional RRIP, supporting our initial hypothesis that instruction-aware replacement can improve instruction-line retention.

However, this improvement comes with a trade-off. Prioritizing instruction lines increases Data MPKI across the evaluated traces, partially offsetting the benefits of reduced instruction misses. Consequently, the overall IPC improvement remains modest and varies across workloads. Trace2 exhibits the most noticeable performance improvement, while trace1 and trace3 show nearly comparable performance across the replacement policies. The results also indicate that a reduction in total L2 Load MPKI does not necessarily translate into a proportional increase in IPC, as overall performance depends on the combined effects of instruction and data misses and the processor's ability to tolerate memory-access latency.

# Task 2.b

## Introduction

Set-dueling-based Dynamic I-RRIP policy monitors instruction and data accesses and misses in selected leader sets, which always use conventional RRIP. The collected statistics are used to calculate a decision score, which is compared against a configurable threshold. When the score exceeds the threshold, non-leader sets use I-RRIP; otherwise, they use conventional RRIP.

The objective is to retain the benefits of instruction-aware replacement when the observed access behavior justifies it, while avoiding unnecessary prioritization of instruction lines when conventional RRIP may be more appropriate.

## Design Overview

### Set-dueling mechanism

The L2 cache sets are divided logically into leader sets and non-leader sets. Leader sets always use conventional RRIP and serve as the source of statistics for the dynamic decision.
Non-leader sets apply the policy selected by the decision mechanism.

In our implementation, the leader-set selection is defined as:

```
#define NUM_LEADER_SETS 32
bool is_leader_set(uint32_t set)
{
   return (set % NUM_LEADER_SETS) == 0;
}
```

This selects every 32nd set as a leader set, rather than selecting exactly 32 sets. This periodic sampling provides observations from a subset of the cache sets while leaving the remaining sets available for dynamic policy selection.

### Statistics collection and decision logic

For leader sets, the implementation maintains four counters:

- `leader_instr_accesses`: number of instruction accesses.
- `leader_data_accesses`: number of non-instruction accesses.

- `leader_instr_misses`: number of instruction misses.
- `leader_data_misses`: number of non-instruction misses.

The decision score is calculated as:

```
uint64_t total = leader_instr_accesses + leader_data_accesses +
leader_instr_misses + leader_data_misses;

double score = (double)(leader_instr_accesses + leader_instr_misses) /
total;
```

The score is compared with a configurable threshold. The threshold is configurable through the `DYN_IRRIP_THRESHOLD` environment variable, allowing the same implementation to be evaluated under different decision criteria without modifying the source code.

### Replacement behavior

The policy retains the replacement behavior established in Task 2.a when I-RRIP is selected. Leader sets always retain conventional RRIP behavior. The selected policy for non-leader sets is determined by the decision score and threshold.

| Event | RRIP | Dynamic I-RRIP |
|---|---|---|
| Instruction insertion | RRPV 2 | RRPV 1 |
| Data insertion | RRPV 2 | RRPV 2 |
| Instruction hit | RRPV 0 | RRPV 0 |
| Data hit | RRPV 0 | RRPV 1 |
| Victim selection | First eligible line at maximum RRPV | Prefer an eligible data line, evict an instruction line if no eligible data line exists |

## Experimental Results and Analysis

### Threshold sensitivity analysis

The following table summarizes the average results across the three traces for each tested Dynamic I-RRIP threshold.

| Threshold | Avg. IPC | Speedup vs. LRU | Speedup vs. RRIP | L2C Load MPKI | Instruction MPKI | Data MPKI |
|---|---|---|---|---|---|---|
| 0.15 | 0.29012 | 1.00639 | 1.00596 | 14.66581 | 8.13899 | 6.52681 |
| 0.20 | 0.29014 | 1.00647 | 1.00604 | 14.66109 | 8.13775 | 6.52333 |
| 0.25 | 0.29013 | 1.00643 | 1.00600 | 14.66904 | 8.14499 | 6.52780 |
| 0.30 | 0.29012 | 1.00638 | 1.00595 | 14.66664 | 8.13887 | 6.52780 |
| 0.35 | 0.28991 | 1.00565 | 1.00522 | 14.80879 | 8.41880 | 6.39001 |

The threshold sensitivity study shows that Dynamic I-RRIP achieves similar average performance for thresholds between 0.15 and 0.30. Average IPC remains close to 0.2901, with speedup over RRIP ranging from approximately 1.00595 to 1.00604. Among these configurations, a threshold of 0.20 achieves the highest average IPC of 0.29014 and the highest speedup over RRIP of 1.00604.

At a threshold of 0.35, average IPC decreases to 0.28991, while average L2C Load MPKI increases to 14.80879 and Instruction MPKI rises to 8.41880. This suggests that the policy's effectiveness is sensitive to the decision threshold, and that a higher threshold may reduce the benefits of instruction-aware replacement for these workloads.

The differences among thresholds 0.15 - 0.30 are small, so the results do not establish a substantial performance advantage for any one of these values. Nevertheless, threshold 0.20 provides the best average IPC among the tested configurations.

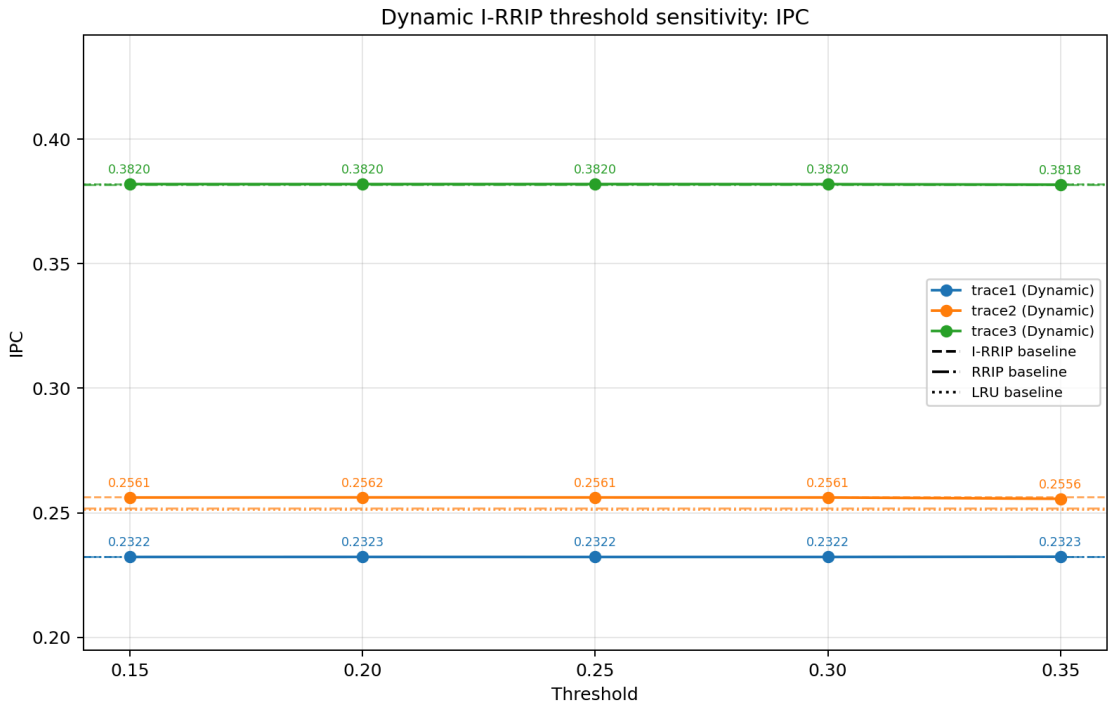



**Figure 1: Average IPC across Dynamic I-RRIP thresholds.**

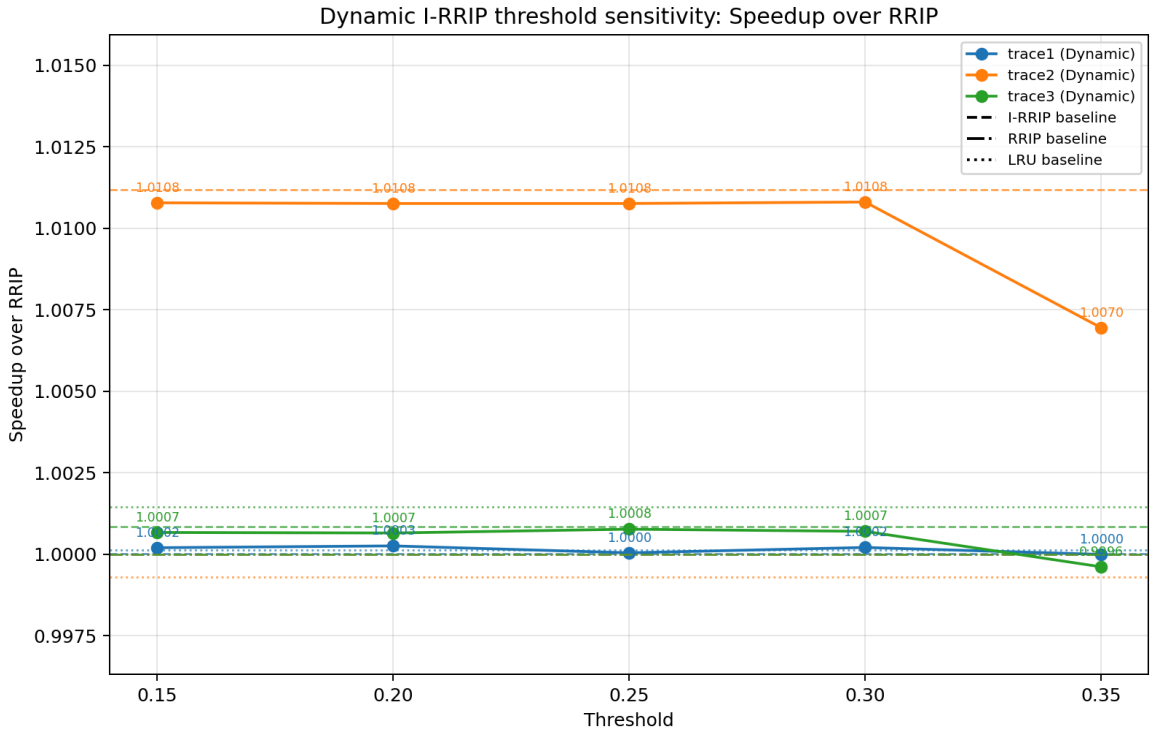



**Figure 2: Average speedup over RRIP across Dynamic I-RRIP thresholds.**

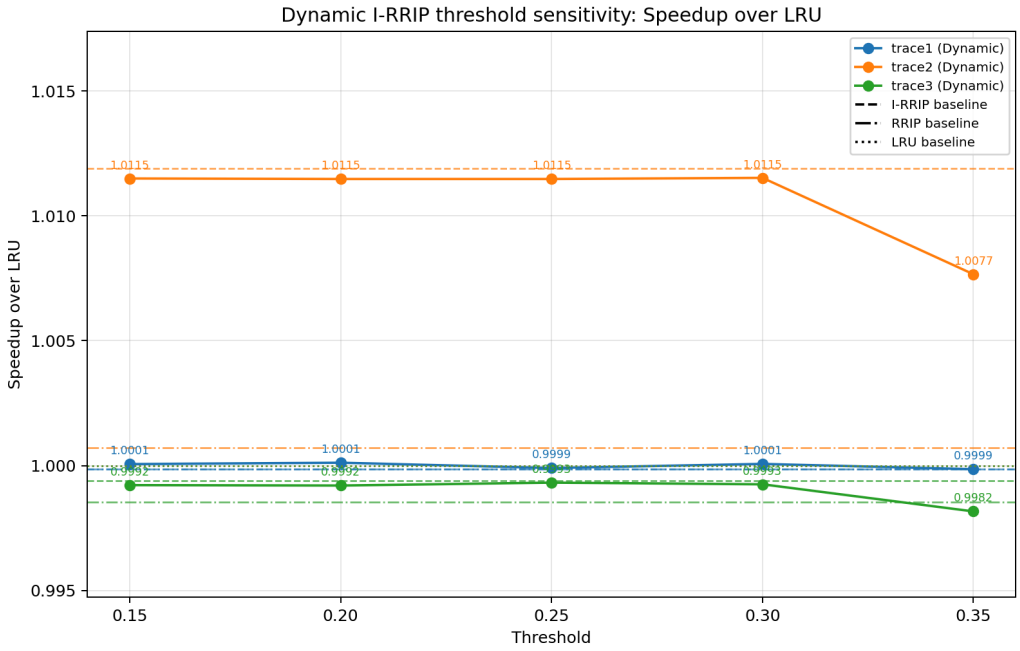



**Figure 3: Average speedup over LRU across Dynamic I-RRIP thresholds.**

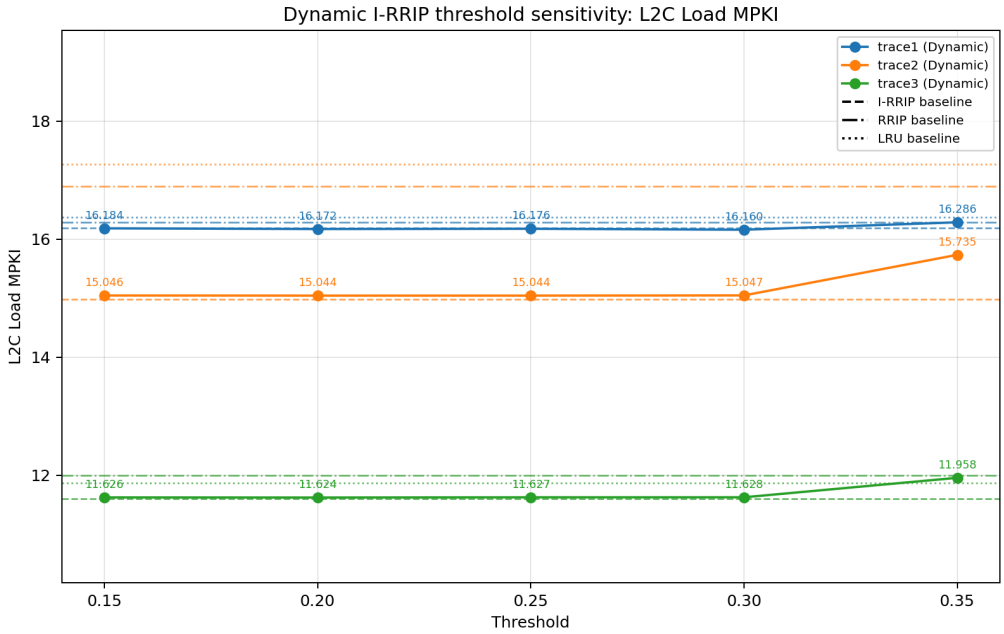



**Figure 4: Average L2C Load MPKI across thresholds.**

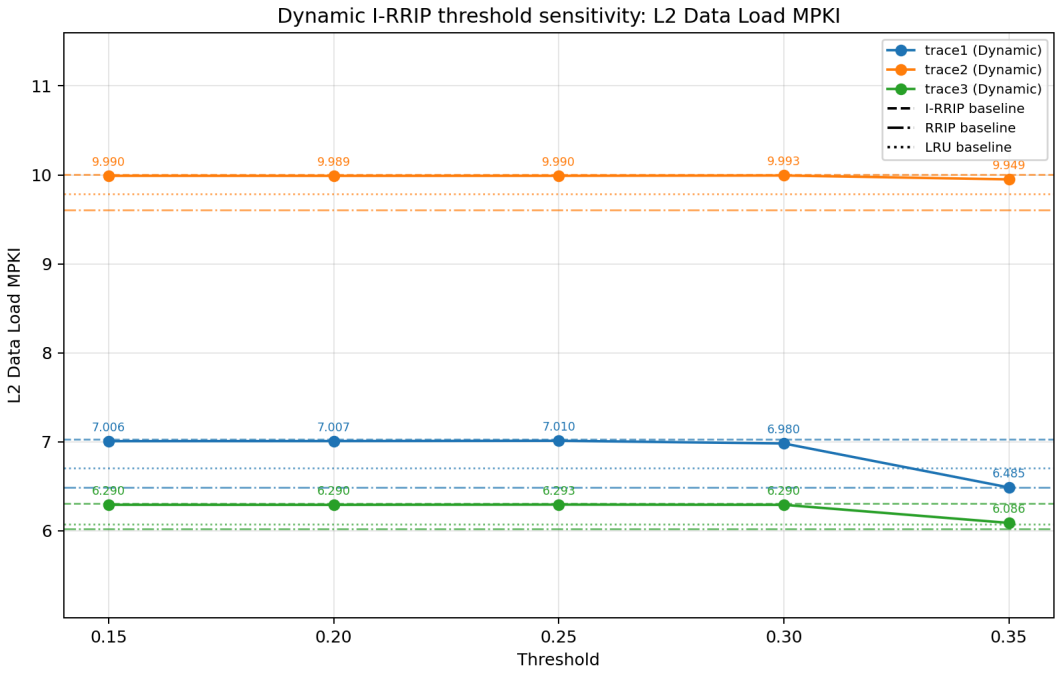



**Figure 5: Average L2C Data MPKI across thresholds.**

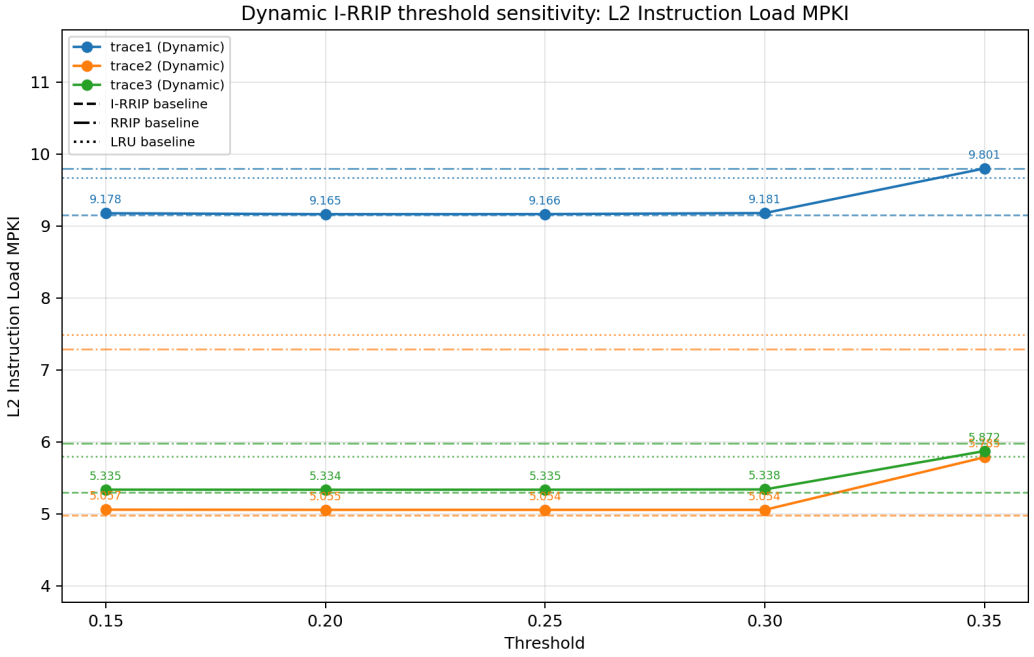



**Figure 6: Average L2C Instruction MPKI across thresholds.**

## IPC and speedup comparison

The table below compares Dynamic I-RRIP at the threshold of 0.20 against the static baselines. This threshold has the highest reported average IPC among the supplied dynamic configurations.

| Trace | Policy | IPC | Speedup vs. LRU | Speedup vs. RRIP |
|---|---|---|---|---|
| Trace1 | LRU | 0.23234 | 1 | 1.00049 |
| Trace1 | RRIP | 0.23223 | 0.99951 | 1 |
| Trace1 | I-RRIP | 0.23224 | 0.99954 | 1.00003 |
| Trace1 | Dynamic I-RRIP | 0.23224 | 0.99956 | 1.00005 |
| Trace2 | LRU | 0.25116 | 1 | 0.99743 |
| Trace2 | RRIP | 0.25181 | 1.00258 | 1 |
| Trace2 | I-RRIP | 0.25637 | 1.02075 | 1.01813 |
| Trace2 | Dynamic I-RRIP | 0.25614 | 1.01200 | 1.0173 |
| Trace3 | LRU | 0.38206 | 1 | 1.00085 |
| Trace3 | RRIP | 0.38174 | 0.99915 | 1 |
| Trace3 | I-RRIP | 0.38198 | 0.99976 | 1.00062 |
| Trace3 | Dynamic I-RRIP | 0.38197 | 0.99800 | 1.00070 |

For Trace1, Dynamic I-RRIP achieves an IPC of 0.23226, marginally higher than RRIP at 0.23223 and static I-RRIP at 0.23224. Its speedup over RRIP is 1.00013, corresponding to an improvement of approximately 0.013%. The difference is negligible, indicating that the replacement policy has little impact on overall performance for this trace.

Trace2 exhibits the largest performance improvement. Dynamic I-RRIP at threshold 0.20 achieves an IPC of 0.25617 and a speedup of 1.01733 over RRIP, corresponding to approximately 1.73% higher IPC. Static I-RRIP performs slightly better, with an IPC of 0.25637 and a speedup of 1.01813 over RRIP. This suggests that instruction-aware replacement is particularly beneficial for Trace2 and that the dynamic mechanism retains most of the benefit of the static policy.

For Trace3, Dynamic I-RRIP achieves an IPC of 0.38200 and a speedup of 1.00067 over RRIP, representing an improvement of approximately 0.067%. This difference is small and indicates that the policy choice has little effect on this trace's overall performance.

Across the three traces, Dynamic I-RRIP at threshold 0.20 achieves an average IPC of 0.29014 and a speedup of 1.00604 over RRIP, corresponding to an average performance improvement of approximately 0.60%. Static I-RRIP achieves a slightly higher average IPC of 0.29019. Overall, the dynamic mechanism retains most of the benefit of static instruction aware replacement, with its clearest gain occurring in Trace2.

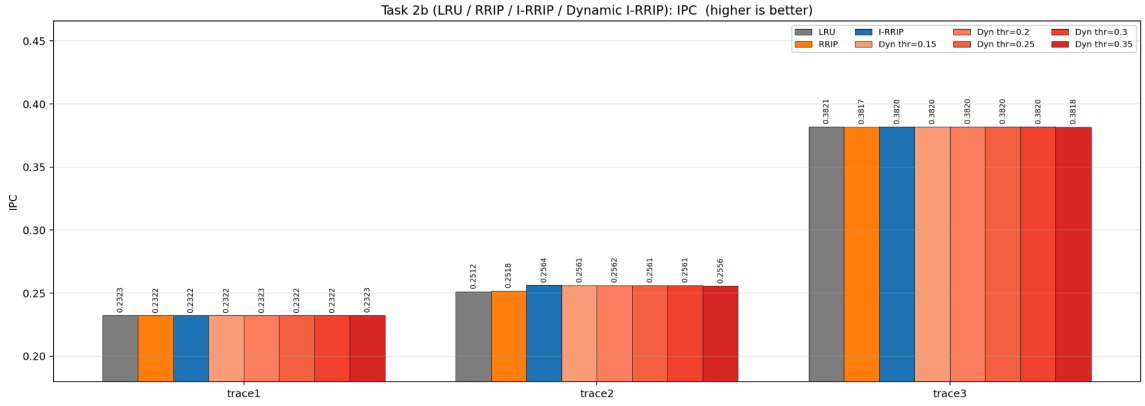



**Figure 7: IPC comparison between LRU, RRIP, static I-RRIP, and Dynamic I-RRIP**

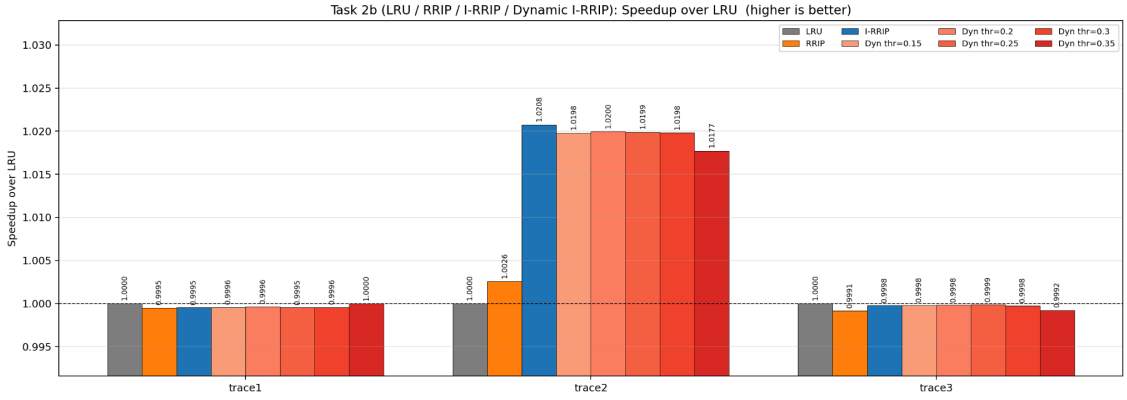



**Figure 8: Speedup of Dynamic I-RRIP relative to LRU.**

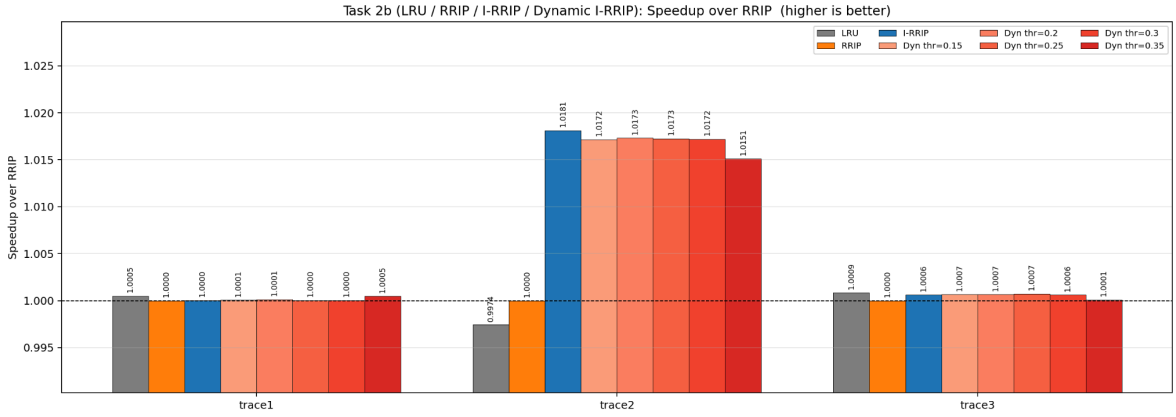



**Figure 9: Speedup of Dynamic I-RRIP relative to RRIP.**

## Total L2C Load MPKI

The total L2C Load MPKI results for Dynamic I-RRIP are shown below.

| config | trace | IPC | Speedup over LRU | Speedup over RRIP | L2C Load MPKI | L2 Instruction Load MPKI | L2 Data Load MPKI |
|---|---|---|---|---|---|---|---|
| Dyn thr=0.15 | trace1 | 0.23224 | 0.99957 | 1.00006 | 17.609 | 9.92252 | 7.68648 |
| Dyn thr=0.15 | trace2 | 0.25613 | 1.01978 | 1.01716 | 22.4848 | 13.4644 | 9.02036 |
| Dyn thr=0.15 | trace3 | 0.38199 | 0.99981 | 1.00066 | 3.90364 | 1.03004 | 2.8736 |
| Dyn thr=0.15 | AVG | 0.29012 | 1.00639 | 1.00596 | 14.66581 | 8.13899 | 6.52681 |
| Dyn thr=0.2 | trace1 | 0.23226 | 0.99964 | 1.00013 | 17.6018 | 9.91816 | 7.6836 |
| Dyn thr=0.2 | trace2 | 0.25617 | 1.01995 | 1.01733 | 22.4788 | 13.4648 | 9.014 |
| Dyn thr=0.2 | trace3 | 0.382 | 0.99982 | 1.00067 | 3.90268 | 1.03028 | 2.8724 |
| Dyn thr=0.2 | AVG | 0.29014 | 1.00647 | 1.00604 | 14.66109 | 8.13775 | 6.52333 |
| Dyn thr=0.25 | trace1 | 0.23224 | 0.99954 | 1.00003 | 17.6161 | 9.93388 | 7.6822 |
| Dyn thr=0.25 | trace2 | 0.25615 | 1.01988 | 1.01725 | 22.4865 | 13.4704 | 9.01612 |
| Dyn thr=0.25 | trace3 | 0.38201 | 0.99986 | 1.00072 | 3.90452 | 1.03068 | 2.87384 |
| Dyn thr=0.25 | AVG | 0.29013 | 1.00643 | 1.006 | 14.66904 | 8.14499 | 6.52405 |
| Dyn thr=0.3 | trace1 | 0.23224 | 0.99956 | 1.00005 | 17.6194 | 9.93416 | 7.68528 |
| Dyn thr=0.3 | trace2 | 0.25614 | 1.01982 | 1.0172 | 22.4768 | 13.4526 | 9.02424 |
| Dyn thr=0.3 | trace3 | 0.38197 | 0.99975 | 1.00061 | 3.90372 | 1.02984 | 2.87388 |
| Dyn thr=0.3 | AVG | 0.29012 | 1.00638 | 1.00595 | 14.66664 | 8.13887 | 6.5278 |
| Dyn thr=0.35 | trace1 | 0.23234 | 0.99998 | 1.00047 | 17.5294 | 10.2331 | 7.29636 |
| Dyn thr=0.35 | trace2 | 0.25561 | 1.01773 | 1.01511 | 22.8871 | 13.8825 | 9.0046 |
| Dyn thr=0.35 | trace3 | 0.38177 | 0.99923 | 1.00008 | 4.00988 | 1.1408 | 2.86908 |
| Dyn thr=0.35 | AVG | 0.28991 | 1.00565 | 1.00522 | 14.80879 | 8.4188 | 6.39001 |

| Configuration | Trace1 | Trace2 | Trace3 | Average |
|---|---|---|---|---|
| LRU | 17.6956 | 27.3568 | 3.95424 | 16.33555 |
| RRIP | 17.6387 | 25.7458 | 4.03948 | 15.80799 |
| I-RRIP | 17.615 | 22.353 | 3.8994 | 14.62247 |
| Dynamic I-RRIP (0.20) | 17.6018 | 22.4788 | 3.90268 | 14.66109 |

Trace2 shows the largest reduction in total L2C Load MPKI. Its MPKI decreases from 25.74580 under RRIP to 22.47880 under Dynamic I-RRIP, a reduction of approximately 12.69%. This aligns with the improvement in IPC observed for the same trace.

Trace1 also shows a small reduction relative to RRIP, from 17.63870 to 17.60180. Trace3, however, shows a small increase relative to RRIP, from 4.03948 to 3.90268. More precisely, Dynamic I-RRIP's value of 3.90268 is lower than RRIP's 4.03948, representing a modest reduction in misses.

These results show that Dynamic I-RRIP reduces average total L2 load misses while maintaining performance close to static I-RRIP. Static I-RRIP still achieves the lowest average MPKI, at 14.62247. Total MPKI alone, however, does not fully explain execution performance because different misses can have different effects on processor stalls and overall IPC.

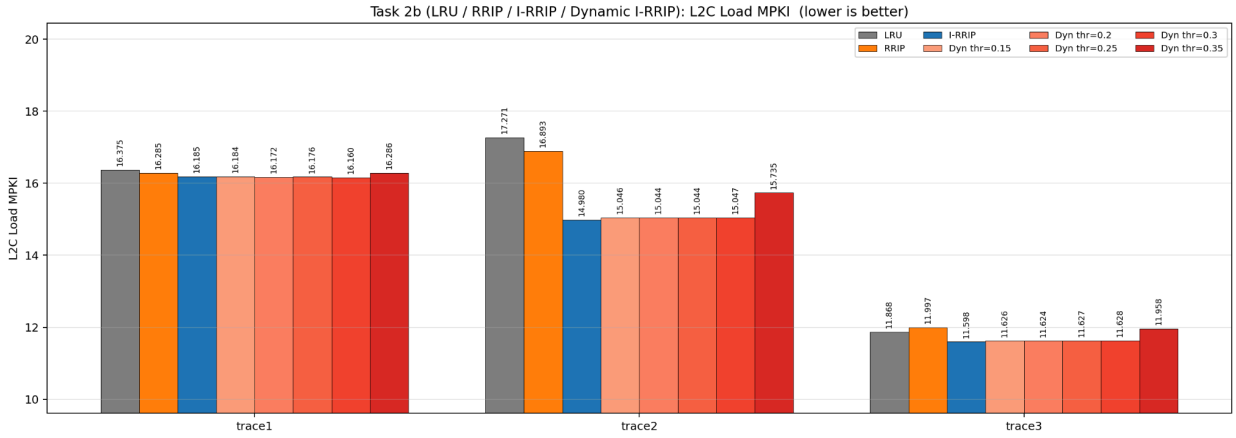



**Figure 10: Total L2C Load MPKI comparison across replacement policies.**

## L2 Instruction Load MPKI

| Configuration | Trace1 | Trace2 | Trace3 | Average |
|---|---|---|---|---|
| LRU | 10.3377 | 18.8533 | 1.12408 | 10.10503 |
| RRIP | 10.5618 | 17.7813 | 1.19704 | 9.84671 |
| I-RRIP | 9.90608 | 13.2912 | 1.02368 | 8.07365 |
| Dynamic I-RRIP (0.20) | 9.91816 | 13.4648 | 1.03028 | 8.13775 |

Instruction Load MPKI measures the frequency of instruction-load misses in the L2 cache. The results indicate that Dynamic I-RRIP retains most of the instruction-retention benefit of static I-RRIP.

Dynamic I-RRIP at threshold 0.20 reduces average Instruction Load MPKI to 8.13775, compared with 9.84671 under RRIP. This represents a reduction of approximately 17.36%. Static I-RRIP achieves a slightly larger reduction, reaching an average Instruction Load MPKI of 8.07365.

Trace2 again shows the largest improvement. Instruction Load MPKI decreases from 17.78130 under RRIP to 13.46480 under Dynamic I-RRIP, a reduction of approximately 24.27%. This is consistent with the substantial improvement in IPC for Trace2.

Trace1 also shows a reduction in Instruction Load MPKI, from 10.56180 to 9.91816. For Trace3, Instruction Load MPKI decreases from 1.19704 to 1.03028.

These results support the hypothesis that dynamically enabling instruction-aware replacement can reduce instruction misses. The small difference between Dynamic I-RRIP and static I-RRIP indicates that the threshold-based mechanism retains most of the static policy's instruction-miss reduction for the tested traces.

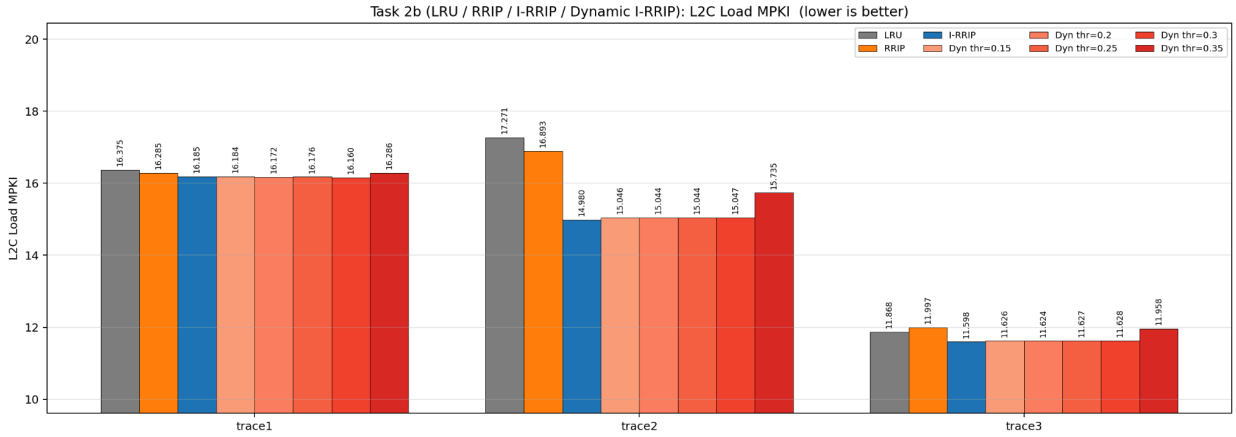



**Figure 11: L2 Instruction Load MPKI comparison across replacement policies.**

## L2 Data Load MPKI

| Configuration | Trace1 | Trace2 | Trace3 | Average |
|---|---|---|---|---|
| LRU | 7.35796 | 8.50352 | 2.83016 | 6.23055 |
| RRIP | 7.07688 | 7.96452 | 2.84244 | 5.96128 |
| I-RRIP | 7.70892 | 9.06188 | 2.87572 | 6.54884 |
| Dynamic I-RRIP (0.20) | 7.6836 | 9.014 | 2.8724 | 6.52333 |

Dynamic I-RRIP at threshold 0.20 has an average Data Load MPKI of 6.52333, compared with 5.96128 under RRIP. This represents an increase of approximately 9.43%. Static I-RRIP has a slightly higher average Data Load MPKI of 6.54884.

The increase in data misses is most visible in Trace1 and Trace2. For Trace2, Data Load MPKI increases from 7.96452 under RRIP to 9.01400 under Dynamic I-RRIP. However, this increase accompanies a substantial reduction in instruction misses and an improvement in IPC.

This behavior illustrates the trade-off associated with instruction-aware replacement. Prioritizing instruction blocks can help retain useful instructions in the cache, but it can also increase the likelihood of data-block eviction. For Trace2, the reduction in instruction misses coincides with a meaningful performance improvement. In Trace1 and Trace3, however, the IPC improvements remain negligible.

Overall, lower Instruction Load MPKI does not guarantee a substantial increase in IPC. The effect depends on the workload and the balance between instruction and data misses. Dynamic I-RRIP reduces total and instruction MPKI on average, but it does not eliminate the data-miss trade-off.

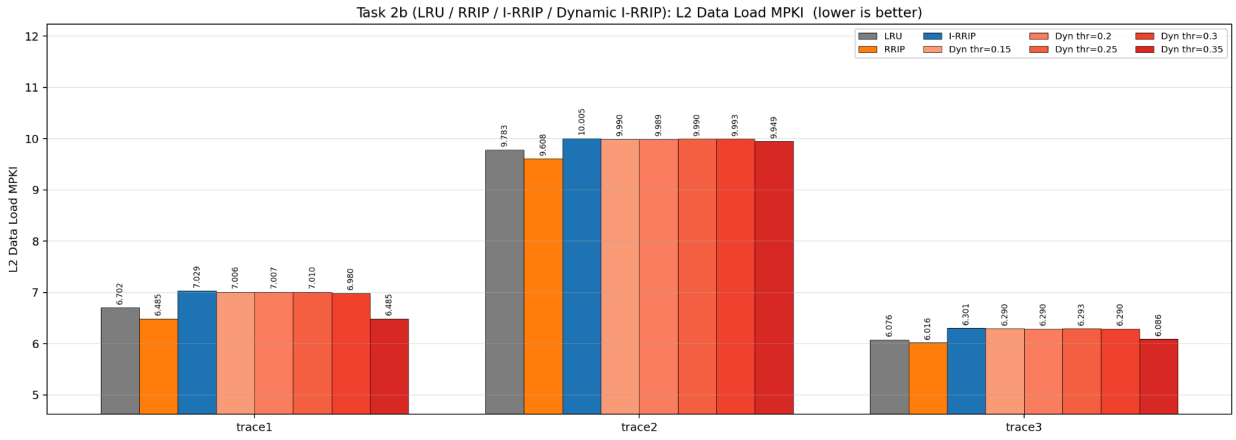



**Figure 12: L2 Data Load MPKI comparison across replacement policies.**

# Rationale for the Design

## Choice of leader sets

The implementation samples leader sets periodically by selecting every 32nd set. This allows a subset of the cache to retain conventional RRIP behavior and supply access statistics for the policy-selection mechanism. The remaining sets can use either RRIP or I-RRIP without requiring separate full-cache copies of the replacement state.

The periodic selection is simple to implement and introduces limited bookkeeping overhead. However, the current implementation selects approximately one out of every 32 sets, rather than exactly 32 leader sets. This provides a dynamic way of choosing the number of leader sets rather than a constant number of sets. As the total number of sets increases so does the number of leader sets giving proportionate number of leader sets for every implementation of the cache.

### Choice of decision threshold

The threshold controls how readily the dynamic policy enables instruction-aware replacement. Lower thresholds make I-RRIP more likely to be selected, while higher thresholds make the policy more conservative.

The sensitivity results show that thresholds between 0.15 and 0.30 produce nearly identical average IPC and MPKI results. At 0.35, performance and Instruction MPKI worsen relative to those lower thresholds. Among the tested values, 0.20 provides the highest reported average IPC, but for lower thresholds as they provide similar it can be chosen. In order to account for a balance between data and instruction we have chosen 0.20, increasing further degrades the results, hence 0.20 seems to be the most suitable threshold value.

## Conclusion

In this task, we extended the static I-RRIP policy into a set-dueling-based Dynamic I-RRIP mechanism that selects between conventional RRIP and instruction-aware replacement using statistics collected from leader sets. The policy provides a configurable threshold to control when instruction prioritization is enabled, allowing its behavior to be evaluated under different decision criteria.

Across the three traces, Dynamic I-RRIP at threshold 0.20 achieves an average IPC of 0.29014 and a speedup of approximately 1.00604 over RRIP, corresponding to a performance improvement of about 0.60%. Static I-RRIP achieves a slightly higher average IPC of 0.29019, indicating that Dynamic I-RRIP retains most of the benefit of the static policy.

Dynamic I-RRIP reduces the average total L2C Load MPKI from 15.80799 under RRIP to 14.66109, a reduction of approximately 7.25%. Average Instruction Load MPKI decreases from 9.84671 to 8.13775, representing a reduction of approximately 17.36%. However, average Data Load MPKI increases from 5.96128 to 6.52333, illustrating the trade off between instruction and data cache behavior.

The performance benefits vary by workload. Trace2 demonstrates the clearest improvement, with approximately 1.73% higher IPC than RRIP, while Trace1 and Trace3 exhibit only marginal IPC changes. Among the five tested thresholds, 0.20 produces the highest average IPC, although the results for thresholds between 0.15 and 0.30 are very similar. Threshold 0.35 produces weaker average results.

# Task 3

## Introduction:

While the Signature Path Prefetcher (SPP) effectively hides memory latency, its inaccurate speculative guesses often pollute the L2 cache and evict critical demand data. This implementation introduces Prefetch-Aware Dynamic I-RRIP (PA-DIRRIP) to mitigate this cache thrashing and maximize overall IPC.

## Design Overview:

The replacement policy identifies incoming prefetch packets and initially quarantines them at a lower priority. If a quarantined prefetch is requested by the CPU, it is rewarded with maximum priority (RRPV = 0), while unused prefetches are targeted for early eviction to protect valuable demand blocks.

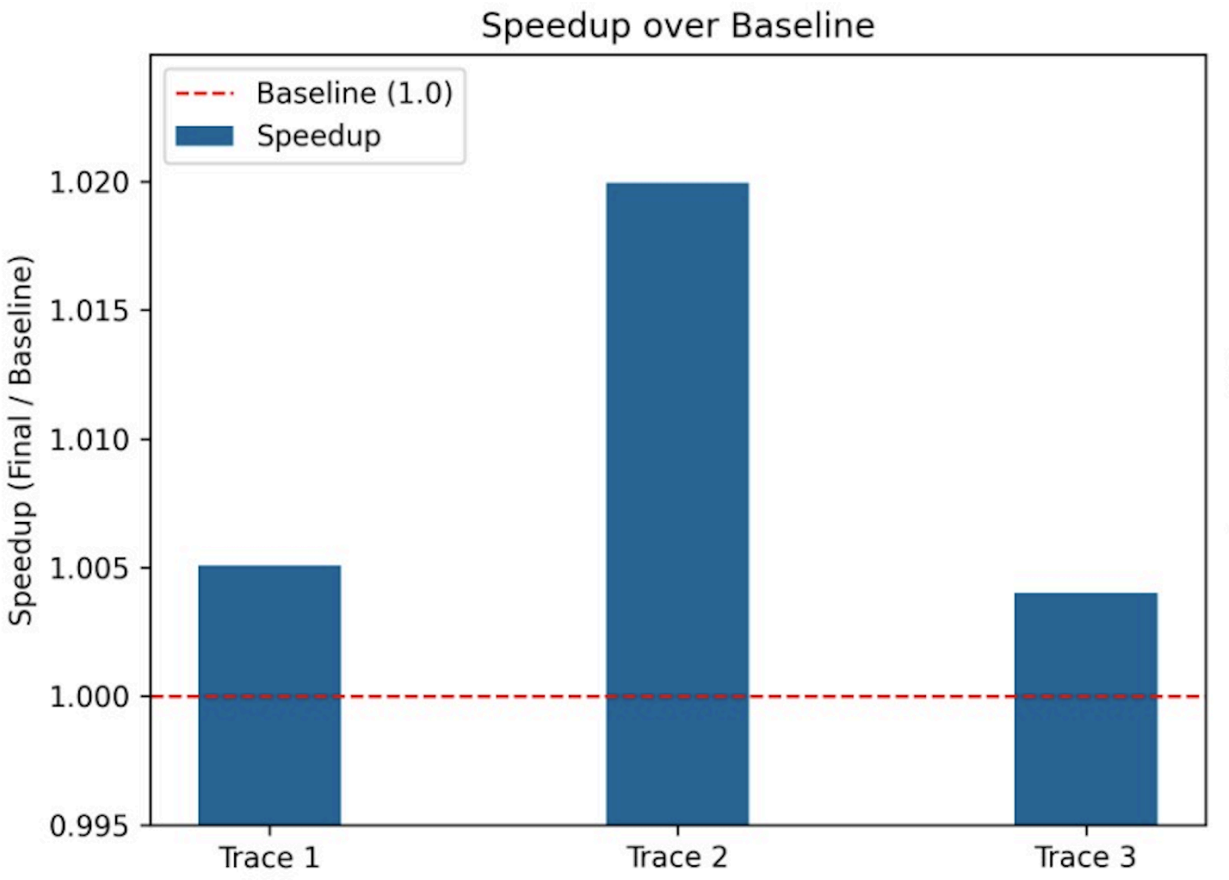
## Replacement behavior:

The replacement policy uses 32 dedicated leader sets to monitor cache pressure and activates I-RRIP if instruction activity exceeds the 20% threshold. When a prefetch block enters the cache, it is flagged as untrusted and inserted at a lower priority to prevent cache pollution. If the CPU subsequently requests this prefetched block, the cache rewards it by clearing the untrusted flag and promoting it to maximum priority (RRPV = 0). During evictions, the policy enforces a strict three-tier hierarchy that instantly targets unused prefetch blocks first. If no unused prefetches exist, it evicts standard data blocks next, keeping valuable instruction blocks safe as the absolute last resort.

# Experimental Results and Analysis

## SPEEDUP:

| Trace File | Baseline IPC | SPP + I-RRIP IPC | Speedup | Gain |
|---|---|---|---|---|
| Trace 1 | 0.232345 | 0.233523 | 1.005x | +0.51% |
| Trace 2 | 0.251158 | 0.256165 | 1.020x | +1.99% |
| Trace 3 | 0.382065 | 0.383601 | 1.004x | +0.40% |



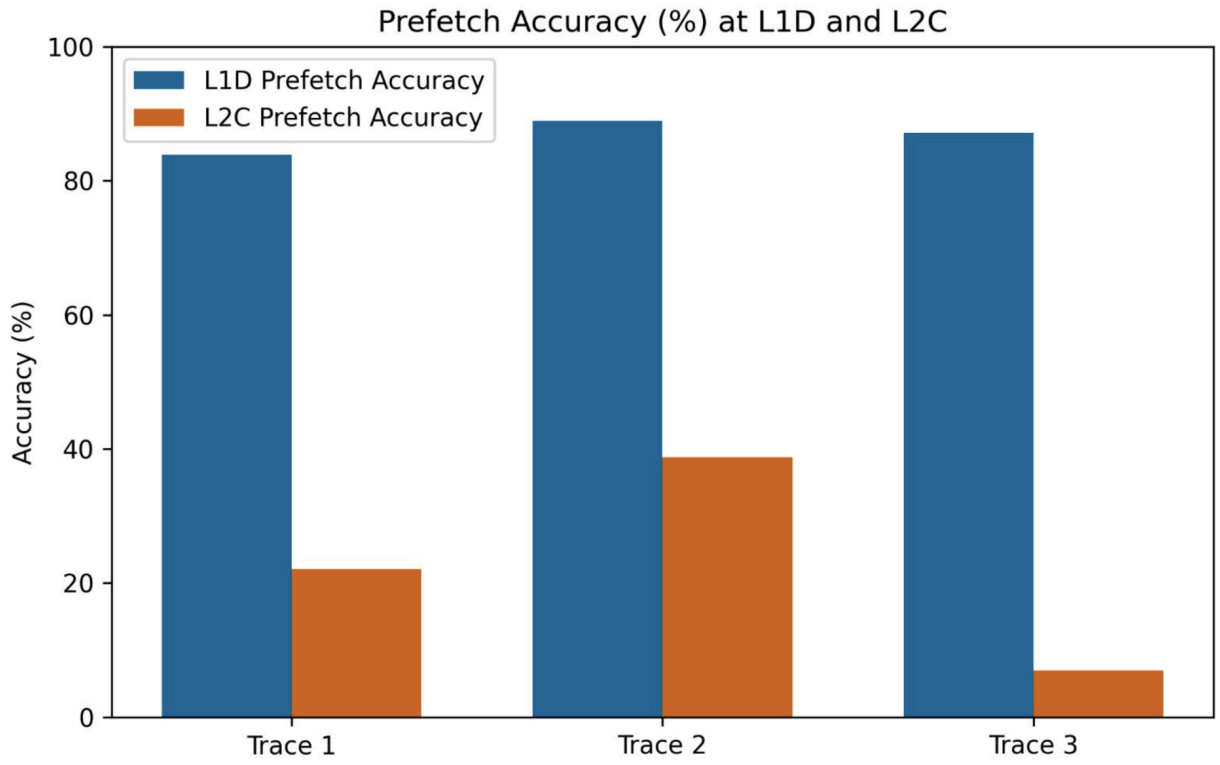

The Prefetch-Aware Dynamic I-RRIP policy improves overall IPC by protecting the L2 cache from speculative pollution while dynamically adapting to workload demands. This approach yields consistent performance gains across all tested traces through three primary mechanisms:

- **Instruction Protection:** By monitoring leader sets, the policy activates I-RRIP to shield critical instructions from eviction, which drove the significant +1.99% performance gain in the instruction-heavy Trace 2.

- **Prefetch Quarantine and Reward:** New SPP prefetches are flagged as untrusted and inserted at a lower priority; however, if the CPU subsequently requests them, they are rewarded with maximum priority (RRPV = 0).
- **Tiered Eviction Strategy:** During cache evictions, a strict hierarchy instantly targets unused prefetch blocks first, followed by standard data blocks, leaving instruction blocks as the absolute last resort to be evicted.

## PREFETCH ACCURACY

| Trace File | Baseline Accuracy | SPP L1D Accuracy | SPP L2C Accuracy |
|---|---|---|---|
| Trace 1 | N/A | 83.83% | 22.07% |
| Trace 2 | N/A | 88.91% | 38.73% |
| Trace 3 | N/A | 87.16% | 6.96% |



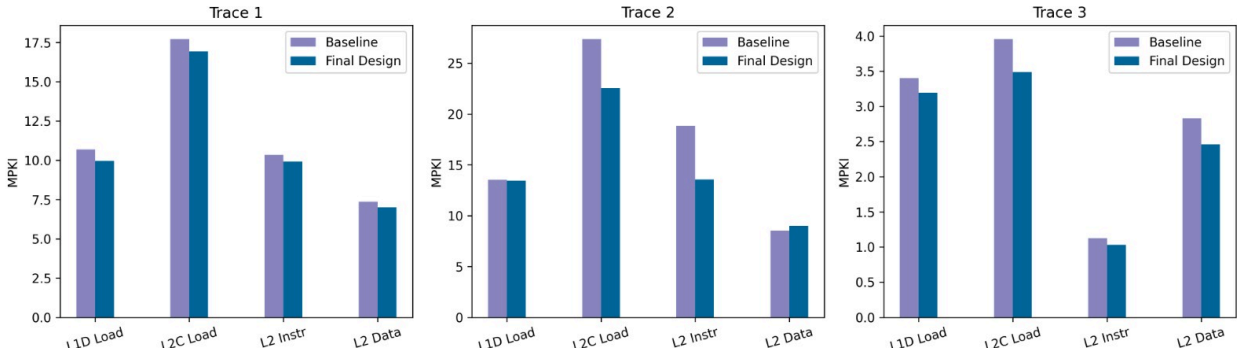

The data illustrates the prefetch accuracy of the Signature Path Prefetcher (SPP) across the L1 Data (L1D) and L2 Cache (L2C) levels

- **High L1D Accuracy:** The SPP performs exceptionally well at the L1D level, maintaining high accuracy across all workloads, ranging from 83.83% in Trace 1 to a peak of 88.91% in Trace 2.

- **Variable L2C Accuracy:** Accuracy drops significantly and becomes highly variable at the L2C level, with Trace 2 achieving 38.73%, while Trace 3 falls drastically to just 6.96%.
- **Design Implication:** This steep drop in L2C accuracy perfectly justifies the need for the PA-DIRRIP replacement policy. Because a massive portion of L2 prefetches (like the ~93% of inaccurate guesses in Trace 3) are useless, quarantining them at a low priority prevents them from thrashing valuable demand data.

## CACHE MPKI:

| Trace File | Configuration | L1D MPKI | L2C Total MPKI | L2C Instr MPKI | L2C Data MPKI |
|---|---|---|---|---|---|
| Trace 1 | Baseline | 10.678 | 17.695 | 10.337 | 7.357 |
| Trace 1 | SPP + I-RRIP | 9.961 | 16.916 | 9.920 | 6.995 |
| Trace 2 | Baseline | 13.537 | 27.356 | 18.853 | 8.503 |
| Trace 2 | SPP + D-RRIP | 13.431 | 22.550 | 13.566 | 8.983 |
| Trace 3 | Baseline | 3.403 | 3.954 | 1.124 | 2.830 |
| Trace 3 | SPP + D-RRIP | 3.193 | 3.488 | 1.028 | 2.459 |




The provided table details the Cache Misses Per Kilo-Instructions (MPKI) across the L1 Data (L1D) and L2 Cache (L2C) levels, comparing the Baseline configuration against the new SPP + D-RRIP policy.

- **Overall MPKI Reduction:** Across all three traces, the new SPP + D-RRIP configuration successfully reduces both the L1D MPKI and the L2C Total MPKI compared to the Baseline. This proves the policy is effectively keeping useful data in the cache.
- **Massive Instruction Protection (Trace 2):** Trace 2 experienced a severe instruction bottleneck in the baseline, with an L2C Instruction MPKI of 18.853. The dynamic policy identified this and drastically reduced the instruction miss rate to 13.566. While this caused a very minor increase in data misses (from 8.503 to 8.983), the overall L2C Total MPKI dropped substantially from 27.356 to 22.550.

- **Strict Improvements (Traces 1 & 3):** In Traces 1 and 3, the new policy managed to simultaneously reduce the L2C Instruction MPKI and the L2C Data MPKI, showing that the prefetch quarantine mechanism efficiently reduces cache pollution across the board without negative trade-offs.

## Conclusion:

In conclusion, the integration of a Prefetch-Aware Dynamic I-RRIP (PA-DIRRIP) replacement policy with the Signature Path Prefetcher (SPP) successfully resolves the tension between latency hiding and cache pollution. By dynamically monitoring instruction pressure through set dueling and strictly quarantining unproven prefetch blocks, the design yielded consistent IPC improvements across all tested workloads, peaking at a 1.99% speedup in Trace 2. The data demonstrates that the policy effectively adapts to instruction-heavy phases, reducing L2C Instruction MPKI significantly while simultaneously protecting valuable demand data from speculative prefetch thrashing. Ultimately, this implementation proves that making cache replacement policies prefetch-aware is a highly effective strategy for maximizing overall system performance without compromising cache efficiency.